

Distributed Database Project Proposal

Due Date: Submitted.

Project ID & Category: #11: Distributed Semi-Join Implementation - Category 2.

1. Project Identity

- **Team Name:** Data Jarvis
- **Team Members:** Tran Truong Thuan
- **Project Title:** Optimizing Distributed Queries with Semi-Join vs. Ship-Whole-Table

2. Objective & Problem Statement

- **The "Why":** Trong môi trường cơ sở dữ liệu phân tán, chi phí truyền tải mạng (Network Communication Cost) thường là nút thắt cổ chai lớn nhất khi thực hiện phép Join giữa các bảng ở các Site khác nhau. Dự án này chứng minh làm thế nào thuật toán Semi-Join giảm thiểu băng thông (Bytes Transferred) so với phương pháp Standard Join (Ship-Whole-Table).
- **Core Logic:** Thực thi phép toán Semi-Join ($R \bowtie S$). Thay vì gửi toàn bộ bảng Assignments (S) sang Node chứa Employees (R), hệ thống chỉ gửi tập hợp các giá trị khóa (EmpID) duy nhất từ S sang R. Trong kiến trúc 3 Site (chứa dữ liệu), Coordinator sẽ phát sóng (broadcast) danh sách khóa này tới các Node chứa phân mảnh của R để lọc song song.

3. Dataset Specification

- **Source:** Dữ liệu tự sinh ngẫu nhiên (Mock Data) bằng thư viện "Faker" của python.
- **Size:**
 - Bảng "Employees": 10,000 dòng.
 - Bảng "Assignments": 50,000 dòng.
- **Schema:**
 - "Employees" (EmpID, Name, Department, Email, Salary).
 - "Assignments" (AssignID, EmpID, ProjectID, HoursLogged).
- **Fragmentation Strategy:** Dữ liệu được lưu trữ phân tán áp dụng **Phân mảnh ngang (Horizontal Frapmentation)**: Node 1 giữ 5,000 dòng Employee đầu tiên, Node 3 giữ 5,000 dòng Employee còn lại. Node 2 giữ toàn bộ bảng Assignment.

4. System Architecture

- **Nodes:** 3 Data Nodes (Node 1, Node 3 for Employee fragments, Node 2 for Assignment) + 1 Coordinator Node.
- **Communication Layer:** HTTP/REST API thông qua framework FastAPI. JSON payload dùng để trao đổi dữ liệu.
- **Storage:** Local CSV files được nạp vào bộ nhớ RAM (bằng Pandas DataFrame) khi hệ thống khởi động để tối ưu tốc độ.

5. Tech Stack & Implementation Plan

- **Programming Language:** Python 3.10+
- **Deployment:** Localhost processes
- **Libraries/Frameworks:** Pandas (xử lý dữ liệu in-memory), FastAPI (networking & API), HTTPX (client requesting).

6. Success Metrics & Analysis

- **Quantitative Metric:**
 - Total Bytes Transferred (Dung lượng truyền tải mạng).
 - Total Execution Time (Thời gian thực thi).
 - Size Reduction Factor (Tỉ lệ giảm thiểu băng thông).
- **The "Failure" Scenario:** Kill Node 2 (Assignment Node) khi Coordinator đang cố gắng gửi request. Coordinator sẽ timeout/catch exception thay vì treo toàn bộ hệ thống (Transaction Aborted).

7. Project Milestones

- **Milestone 1:** Thiết lập môi trường và tạo script sinh dữ liệu ngẫu nhiên (10k dòng Employee chia làm 2 file, 50k dòng Assignment).
- **Milestone 2:** Xây dựng Node 1, Node 2 và Node 3 với các API đọc và thực thi thuật toán phân tán.
- **Milestone 3:** Xây dựng Coordinator, tích hợp đo lường chi phí mạng (Cost Model), kiểm thử "Failure Scenario" và viết Design Document.